Midterm Output

Program Documentation — MSEUF Lucena

Student	Tolete, Allen Jerrome M.
Subject	CP102 — M001
School	MSEUF BS Computer Science
Semester	2nd Semester, Academic Year 2024–2025 | Midterms

Part I: CSV Record Management System

Program: AI Job Displacement Record Manager with Tkinter UI

I. Program Description

This program is a CSV-based Record Management System developed for the Database Exam activity. It is built using Python's Tkinter library for the graphical user interface and follows an Object-Oriented Programming (OOP) architecture. The system is themed after the university colors

The dataset used is the AI Job Replacement 2020–2026 dataset from Kaggle, which contains 15,000 records across 20 columns covering automation risk, salary impact, AI adoption levels, and reskilling urgency across industries and countries. The program allows users to manage a separate custom CSV file of their own records while also being able to browse and filter the main dataset.

Student	Tolete, Allen Jerrome M.
Course	BS Computer Science — CP102 M001
Language	Python 3
Libraries	tkinter, csv, pandas, matplotlib, os, datetime
Dataset	ai_job_replacement_2020_2026_v2.csv (15,000 rows)
Records File	custom_records.csv (created automatically)

The application window is organized into four tabs:

- **My Records** — Add, delete, search, and display custom records saved to CSV.
- **View Dataset** — Browse all 15,000 rows of the Kaggle dataset with Industry and Year filters.
- **Data Analysis** — Five analytical questions answered live using pandas with color-coded output.
- **Charts** — Five selectable chart types rendered using matplotlib embedded in Tkinter.

II. Operational Logic

The program is structured around four OOP classes. Each class has a single responsibility, following the Separation of Concerns design principle.

Class	Role	Key Methods
`JobRecord`	`Data model — stores one record's fields`	`__init__, to_dict, from_dict`

CSVRepository	File I/O — reads/writes custom_records.csv	load_all, save_all, append_record, id_exists
RecordManager	Business logic — add, search, delete	add_record, get_all_records, search_by_id, search_by_name, delete_record
App (tk.Tk)	GUI controller — builds all tabs and widgets	_build_header, _build_tabs, _run_analysis, _render_chart

A. Library Imports

Library Imports

```
import os                # File path handling
import csv               # Reading and writing CSV files
import tkinter as tk     # Main GUI framework
from tkinter import ttk, messagebox, filedialog
from datetime import datetime  # Auto-timestamp for new records
import pandas as pd      # Dataset loading and analysis
import matplotlib        # Chart rendering (embedded in Tkinter)
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
```

B. File Path Setup

All CSV files are located in the same directory as the script. os.path.dirname and os.path.join are used to build absolute paths so the program works regardless of where it is run from.

File Path Constants

```
script_dir   = os.path.dirname(os.path.abspath(__file__))
DATASET_FILE = os.path.join(script_dir, "ai_job_replacement_2020_2026_v2.csv")
RECORDS_FILE = os.path.join(script_dir, "custom_records.csv")
```

C. JobRecord Class

JobRecord is the data model class. Each instance represents one row in the custom CSV. It stores ten fields and provides to_dict() for writing to CSV and from_dict() as a class method for reading from CSV.

JobRecord Class

```
class JobRecord:
    FIELDS = ["ID", "Job Role", "Industry", "Country",
              "Year", "Risk %", "AI Score", "Salary (USD)", "Notes", "Date Added"]

    def __init__(self, rid, job_role, industry, country,
                 year, risk, ai_score, salary, notes='', date_added=None):
        self.rid        = str(rid).strip()
        self.job_role   = str(job_role).strip()
        # ... (all fields stored as stripped strings)
        self.date_added = date_added or datetime.now().strftime('%Y-%m-%d %H:%M')

    def to_dict(self):      # converts to dict for CSV writing
        return {"ID": self.rid, "Job Role": self.job_role, ...}

    @classmethod
    def from_dict(cls, d):  # creates instance from a CSV row dict
        return cls(d["ID"], d["Job Role"], ...)
```

D. CSVRepository Class

CSVRepository handles all direct file operations on custom_records.csv. It automatically creates the file with a header row if it does not exist, and provides methods for loading all records, saving all records at once (used for deletions), and appending a single new record.

CSVRepository Class

```
class CSVRepository:
    def __init__(self, filepath):
        self.filepath = filepath
        self._create_if_not_exists()   # creates CSV with header if missing

    def load_all(self):                # returns list of JobRecord objects
        with open(self.filepath, 'r', newline='', encoding='utf-8') as f:
            reader = csv.DictReader(f)
            return [JobRecord.from_dict(row) for row in reader]

    def append_record(self, record):   # adds one row without rewriting the file
        with open(self.filepath, 'a', newline='', encoding='utf-8') as f:
            csv.DictWriter(f, fieldnames=JobRecord.FIELDS).writerow(record.to_dict())

    def save_all(self, records):       # rewrites entire file (used after deletion)
        ...

    def id_exists(self, rid):          # checks for duplicate IDs before adding
        return any(r.rid == str(rid) for r in self.load_all())
```

E. RecordManager Class

RecordManager contains the business logic. It uses CSVRepository for all file operations and provides clean methods for the GUI to call. It prevents duplicate IDs, handles search by ID or by name/country, and manages deletion.

RecordManager Class

```
class RecordManager:
    def add_record(self, rid, job_role, ...):
        if self.repo.id_exists(rid):
            return False, f"ID '{rid}' already exists."
        self.repo.append_record(JobRecord(rid, job_role, ...))
        return True, f"Record added! (ID: {rid})"

    def get_all_records(self):          # returns all records as a list
    def search_by_id(self, rid):        # returns one record or None
    def search_by_name(self, query):    # returns list (partial match on role/country)
    def delete_record(self, rid):       # removes record, rewrites CSV
```

F. App Class (Main GUI)

App inherits from tk.Tk and serves as the main application controller. It initializes the backend objects, loads the dataset, sets up ttk styles, builds the header, and constructs all four tabs.

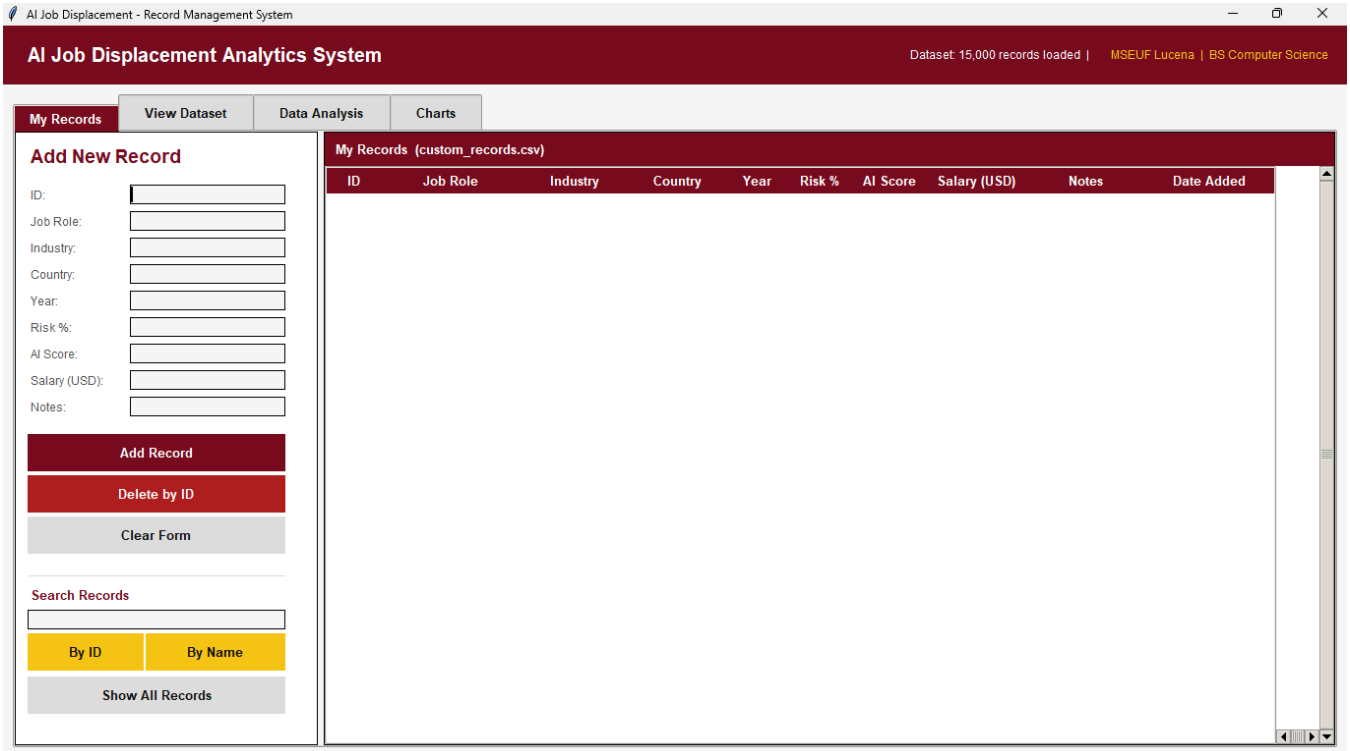
- **`_build_records_tab()`** — Left panel form + right panel Treeview table. Handles add, delete, and search operations.
- **`_build_dataset_tab()`** — Loads full 15,000-row dataset into a scrollable Treeview with Industry and Year filter dropdowns.
- **`_build_analysis_tab()`** — Text output area with color-coded results for 5 pandas analytical questions.
- **`_build_charts_tab()`** — Five chart buttons. Charts rendered via matplotlib FigureCanvasTkAgg embedded in the Tkinter window.

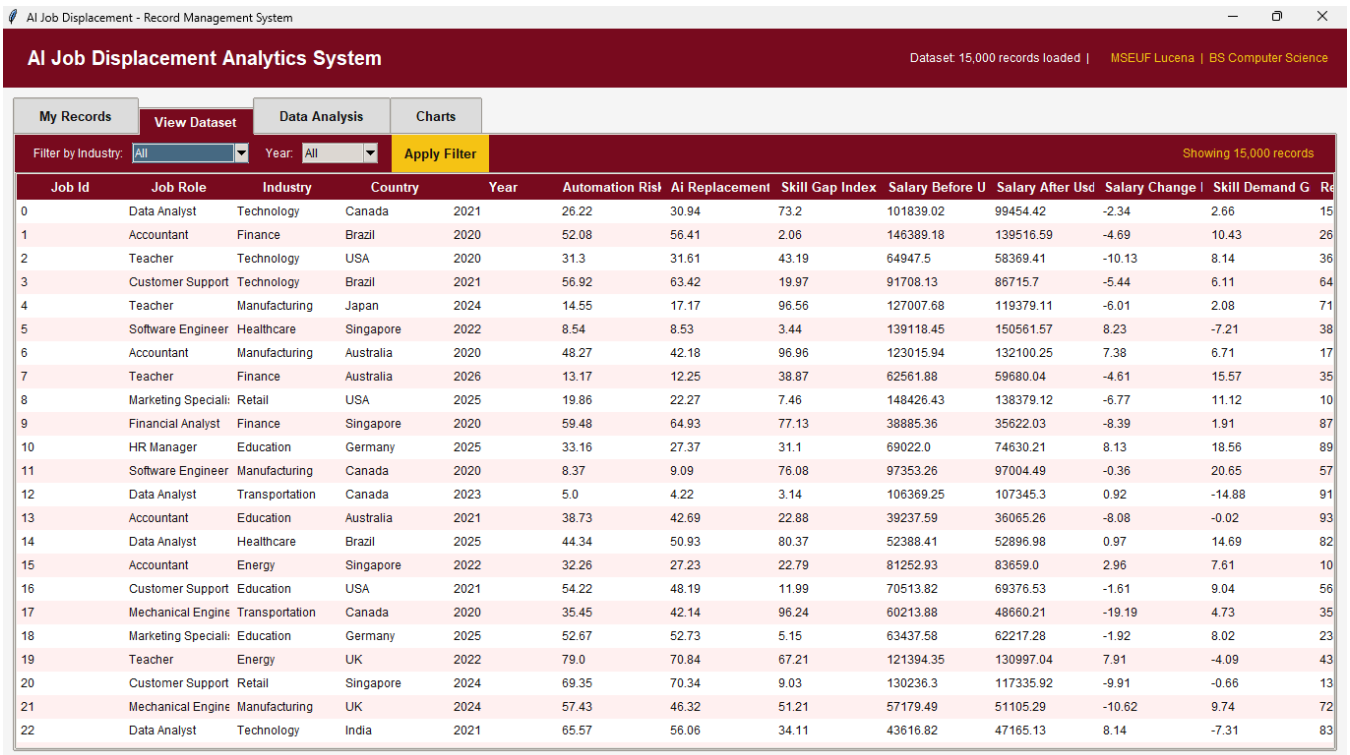
III. Source Code

The complete source code is listed below.

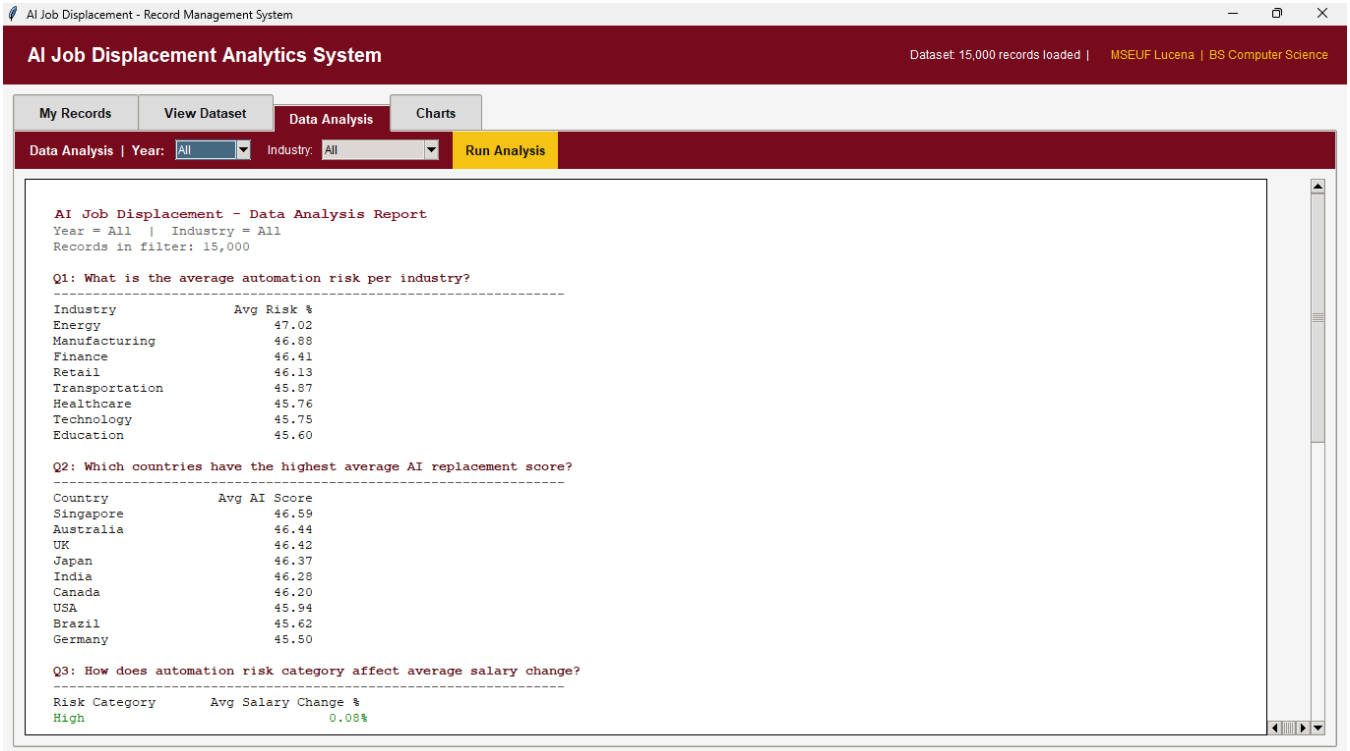
```
# Activity 4 - Accessing File (CSV) with GUI
# Student: Allen | Course: BS Computer Science | School: MSEUF Lucena

import os, csv, tkinter as tk
from tkinter import ttk, messagebox, filedialog
from datetime import datetime
import pandas as pd
import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

script_dir   = os.path.dirname(os.path.abspath(__file__))
DATASET_FILE = os.path.join(script_dir, "ai_job_replacement_2020_2026_v2.csv")
RECORDS_FILE = os.path.join(script_dir, "custom_records.csv")

MAROON='#7B0D1E'; MAROON2='#5C0A16'; YELLOW='#F5C518'
WHITE='#FFFFFF'; LGRAY='#F5F5F5'; GRAY='#DDDDDD'
FONT=('Arial',10); FONT_BOLD=('Arial',10,'bold')
FONT_TITLE=('Arial',14,'bold'); FONT_SMALL=('Arial',9)

class JobRecord:
    FIELDS = ["ID","Job Role","Industry","Country","Year",
              "Risk %","AI Score","Salary (USD)","Notes","Date Added"]
    def __init__(self, rid, job_role, industry, country,
                 year, risk, ai_score, salary, notes='', date_added=None):
        self.rid=str(rid).strip(); self.job_role=str(job_role).strip()
        self.industry=str(industry).strip(); self.country=str(country).strip()
        self.year=str(year).strip(); self.risk=str(risk).strip()
        self.ai_score=str(ai_score).strip(); self.salary=str(salary).strip()
        self.notes=str(notes).strip()
        self.date_added=date_added or datetime.now().strftime('%Y-%m-%d %H:%M')
    def to_dict(self):
        return {"ID":self.rid,"Job Role":self.job_role,"Industry":self.industry,
                "Country":self.country,"Year":self.year,"Risk %":self.risk,
                "AI Score":self.ai_score,"Salary (USD)":self.salary,
                "Notes":self.notes,"Date Added":self.date_added}
    @classmethod
    def from_dict(cls,d):
        return cls(d["ID"],d["Job Role"],d["Industry"],d["Country"],
                   d["Year"],d["Risk %"],d["AI Score"],d["Salary (USD)"],
                   d.get("Notes",""),d.get("Date Added",""))

class CSVRepository:
    def __init__(self,filepath):
        self.filepath=filepath; self._create_if_not_exists()
    def _create_if_not_exists(self):
        if not os.path.exists(self.filepath):
            with open(self.filepath,'w',newline='',encoding='utf-8') as f:
                csv.DictWriter(f,fieldnames=JobRecord.FIELDS).writeheader()
    def load_all(self):
        with open(self.filepath,'r',newline='',encoding='utf-8') as f:
            return [JobRecord.from_dict(r) for r in csv.DictReader(f)]
    def save_all(self,records):
        with open(self.filepath,'w',newline='',encoding='utf-8') as f:
            w=csv.DictWriter(f,fieldnames=JobRecord.FIELDS)
            w.writeheader()
            for r in records: w.writerow(r.to_dict())
    def append_record(self,record):
        with open(self.filepath,'a',newline='',encoding='utf-8') as f:
            csv.DictWriter(f,fieldnames=JobRecord.FIELDS).writerow(record.to_dict())
```

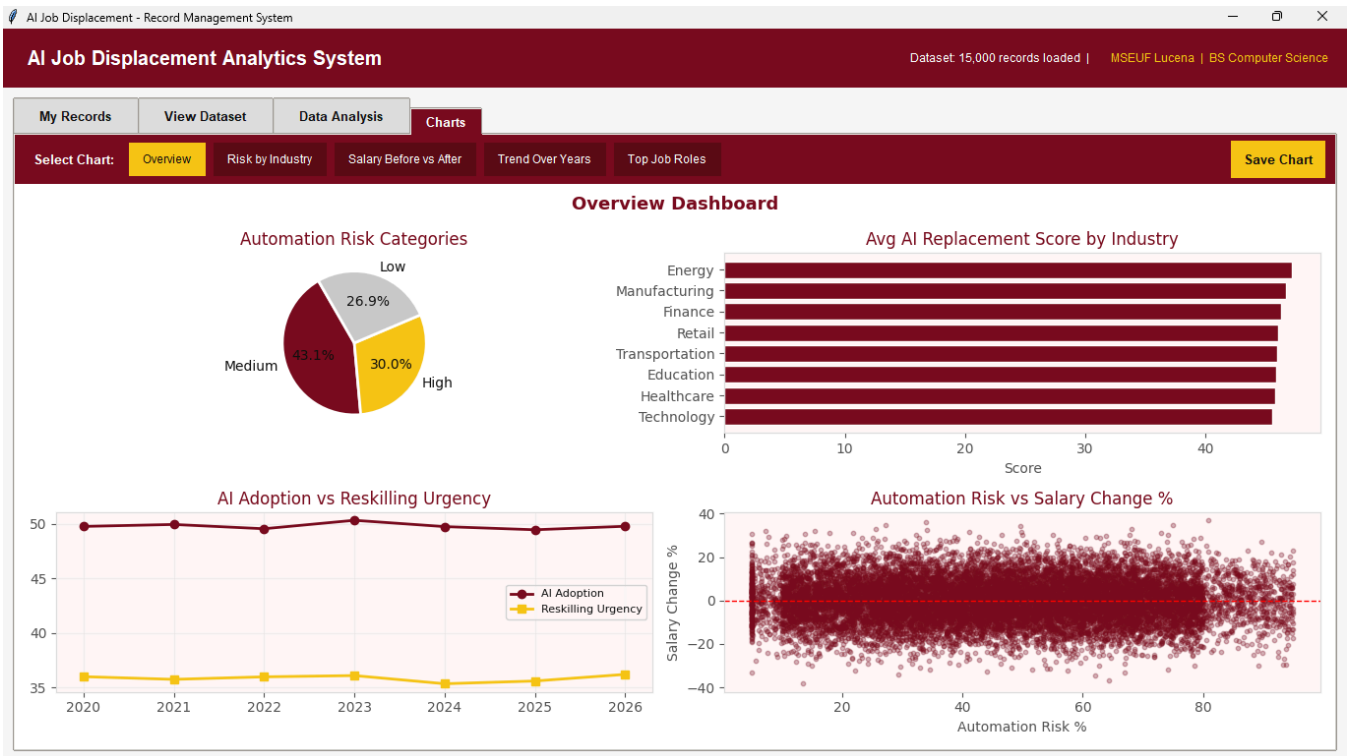
```
    def id_exists(self,rid):
        return any(r.rid==str(rid) for r in self.load_all())

class RecordManager:
    def __init__(self,repo): self.repo=repo
    def add_record(self,rid,job_role,industry,country,year,risk,ai_score,salary,notes):
        if self.repo.id_exists(rid): return False,f"ID '{rid}' already exists."
        self.repo.append_record(JobRecord(rid,job_role,industry,country,
                                          year,risk,ai_score,salary,notes))
        return True,f'Record added! (ID: {rid})'
    def get_all_records(self): return self.repo.load_all()
    def search_by_id(self,rid):
        for r in self.repo.load_all():
            if r.rid==str(rid): return r
        return None
    def search_by_name(self,query):
        q=query.lower()
        return [r for r in self.repo.load_all()
                if q in r.job_role.lower() or q in r.country.lower()]
    def delete_record(self,rid):
        records=self.repo.load_all()
        filtered=[r for r in records if r.rid!=str(rid)]
        if len(filtered)==len(records): return False,f"No record with ID '{rid}'."
        self.repo.save_all(filtered); return True,f"Record '{rid}' deleted."

class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title('AI Job Displacement - Record Management System')
        self.geometry('1100x700'); self.configure(bg=LGRAY)
        self.repo=CSVRepository(RECORDS_FILE)
        self.manager=RecordManager(self.repo)
        self.df=self._load_dataset()
        self._setup_styles(); self._build_header(); self._build_tabs()
    # ... (see full source file for complete tab implementations)

if __name__ == '__main__':
    app = App()
    app.mainloop()
```





Job Id	Job Role	Industry	Country	Year	Automation Risk	Ai Replacement	Skill Gap Index	Salary Before U	Salary After Usd	Salary Change	Skill Demand G	Re
0	Data Analyst	Technology	Canada	2021	26.22	30.94	73.2	101839.02	99454.42	-2.34	2.66	15
1	Accountant	Finance	Brazil	2020	52.08	56.41	2.06	146389.18	139516.59	-4.69	10.43	26
2	Teacher	Technology	USA	2020	31.3	31.61	43.19	64947.5	58369.41	-10.13	8.14	36
3	Customer Support	Technology	Brazil	2021	56.92	63.42	19.97	91708.13	86715.7	-5.44	6.11	64
4	Teacher	Manufacturing	Japan	2024	14.55	17.17	96.56	127007.68	119379.11	-6.01	2.08	71
5	Software Engineer	Healthcare	Singapore	2022	8.54	8.53	3.44	139118.45	150561.57	8.23	-7.21	38
6	Accountant	Manufacturing	Australia	2020	48.27	42.18	96.96	123015.94	132100.25	7.38	6.71	17
7	Teacher	Finance	Australia	2026	13.17	12.25	38.87	62561.88	59680.04	-4.61	15.57	35
8	Marketing Speciali	Retail	USA	2025	19.86	22.27	7.46	148426.43	138379.12	-6.77	11.12	10
9	Financial Analyst	Finance	Singapore	2020	59.48	64.93	77.13	38885.36	35622.03	-8.39	1.91	87
10	HR Manager	Education	Germany	2025	33.16	27.37	31.1	69022.0	74630.21	8.13	18.56	89
11	Software Engineer	Manufacturing	Canada	2020	8.37	9.09	76.08	97353.26	97004.49	-0.36	20.65	57
12	Data Analyst	Transportation	Canada	2023	5.0	4.22	3.14	106369.25	107345.3	0.92	-14.88	91
13	Accountant	Education	Australia	2021	38.73	42.69	22.88	39237.59	36065.26	-8.08	-0.02	93
14	Data Analyst	Healthcare	Brazil	2025	44.34	50.93	80.37	52388.41	52896.98	0.97	14.69	82
15	Accountant	Energy	Singapore	2022	32.26	27.23	22.79	81252.93	83659.0	2.96	7.61	10
16	Customer Support	Education	USA	2021	54.22	48.19	11.99	70513.82	69376.53	-1.61	9.04	56
17	Mechanical Engine	Transportation	Canada	2020	35.45	42.14	96.24	60213.88	48660.21	-19.19	4.73	35
18	Marketing Speciali	Education	Germany	2025	52.67	52.73	5.15	63437.58	62217.28	-1.92	8.02	23
19	Teacher	Energy	UK	2022	79.0	70.84	67.21	121394.35	130997.04	7.91	-4.09	43
20	Customer Support	Retail	Singapore	2024	69.35	70.34	9.03	130236.3	117335.92	-9.91	-0.66	13
21	Mechanical Engine	Manufacturing	UK	2024	57.43	46.32	51.21	57179.49	51105.29	-10.62	9.74	72
22	Data Analyst	Technology	India	2021	65.57	56.06	34.11	43616.82	47165.13	8.14	-7.31	83


AI Job Displacement - Record Management System
AI Job Displacement Analytics System
Dataset: 15,000 records loaded | MSEUF Lucena | BS Computer Science
My Records
View Dataset
Data Analysis
Charts
Data Analysis | Year: All
Industry: All
Run Analysis
AI Job Displacement - Data Analysis Report
Year = All | Industry = All
Records in filter: 15,000
Q1: What is the average automation risk per industry?
Industry Avg Risk %
Energy 47.02
Manufacturing 46.88
Finance 46.41
Retail 46.13
Transportation 45.87
Healthcare 45.76
Technology 45.75
Education 45.60
Q2: Which countries have the highest average AI replacement score?
Country Avg AI Score
Singapore 46.59
Australia 46.44
UK 46.42
Japan 46.37
India 46.28
Canada 46.20
USA 45.94
Brazil 45.62
Germany 45.50
Q3: How does automation risk category affect average salary change?
Risk Category Avg Salary Change %
High 0.08%


AI Job Displacement - Record Management System
AI Job Displacement Analytics System
Dataset: 15,000 records loaded | MSEUF Lucena | BS Computer Science
My Records
View Dataset
Data Analysis
Charts
Select Chart:
Overview
Risk by Industry
Salary Before vs After
Trend Over Years
Top Job Roles
Save Chart
Overview Dashboard
Automation Risk Categories
Low
26.9%
Medium
43.1%
30.0%
High
Avg AI Replacement Score by Industry
Energy
Manufacturing
Finance
Retail
Transportation
Education
Healthcare
Technology
0
10
20
30
40
Score
AI Adoption vs Reskilling Urgency
50
45
40
35
2020
2021
2022
2023
2024
2025
2026
AI Adoption
Reskilling Urgency
Automation Risk vs Salary Change %
Salary Change %
40
20
0
−20
−40
20
40
60
80
Automation Risk %

IV. Areas for Improvement

- **Input Validation** — The form currently accepts any text in every field. Validation should ensure the Year field only accepts four-digit numbers, Risk % accepts values between 0 and 100, and Salary only accepts positive numbers.

- **Edit Record Functionality** — Currently, users can only add or delete records. An edit feature would allow users to update existing entries without deleting and re-adding them.

- **Duplicate ID Feedback** — While RecordManager already checks for duplicate IDs, the error message could be more prominent in the UI — perhaps a pop-up dialog instead of a small status label.

- **Export to Excel or PDF** — Adding a button to export the custom records table to an Excel or PDF file would make the system more useful in real scenarios.

- **Pagination for Dataset Tab** — Loading all 15,000 rows at once in the View Dataset tab can be slow. Pagination or lazy loading would improve performance significantly.

V. Reflection

Developing this Record Management System was a practical exercise in applying Object-Oriented Programming to a real-world use case. By separating the program into distinct classes — JobRecord for data, CSVRepository for file handling, RecordManager for logic, and App for the GUI — I was able to keep each component focused on a single responsibility. This made the code easier to debug and expand.

Working with the CSV module for file persistence was straightforward, but I learned the importance of always opening files in the correct mode (append vs. write) to avoid accidentally overwriting data. The os.path approach for file paths was also a key lesson — hardcoded paths would break the program on other machines.

The Tkinter GUI required careful layout planning. Using ttk.Notebook for tabs and ttk.Treeview for the records table made the interface feel more professional. Embedding matplotlib charts directly into the Tkinter window using FigureCanvasTkAgg was particularly rewarding — it showed how two major Python libraries can work together seamlessly inside a single application.

Part II: Data Analysis & Visualization

Program: Jupyter Notebook with pandas and hvplot

I. Program Description

Part II is a Jupyter Notebook that performs data analysis and visualization on the AI Job Replacement 2020-2026 dataset using pandas and hvplot. The notebook addresses five required analytical questions plus two bonus questions, each paired with an hvplot chart. It uses at least three different chart types as required by the instructions.

Notebook File	Database_Exam_Part_2_-_Analysis_using_HvPlot.ipynb
Dataset	ai_job_replacement_2020_2026_v2.csv (15,000 rows, 20 columns)
Tools	Python 3, pandas, hvplot
Chart Types	Bar, Horizontal Bar, Line (overlaid), Scatter, Histogram
Questions	5 required + 2 bonus = 7 total
Install	pip install hvplot pandas

Key columns used in the analysis: industry, country, automation_risk_percent, ai_replacement_score, salary_before_usd, salary_after_usd, salary_change_percent, ai_adoption_level, reskilling_urgency_score, automation_risk_category, and year.

II. Operational Logic

A. Library Imports and Dataset Loading

The notebook begins by importing os, pandas, and hvplot.pandas. Importing hvplot.pandas activates the .hvplot accessor on all DataFrames, allowing charts to be generated with a single method call.

Setup Cell

```
import os
import pandas as pd
import hvplot.pandas   # activates df.hvplot accessor

script_dir = os.path.dirname(os.path.abspath('__file__'))
filename = os.path.join(script_dir, "ai_job_replacement_2020_2026_v2.csv")
df = pd.read_csv(filename)
df.head()      # preview first 5 rows
df.info()      # column types and null counts
df.describe()  # summary statistics
```

B. Analytical Questions Summary

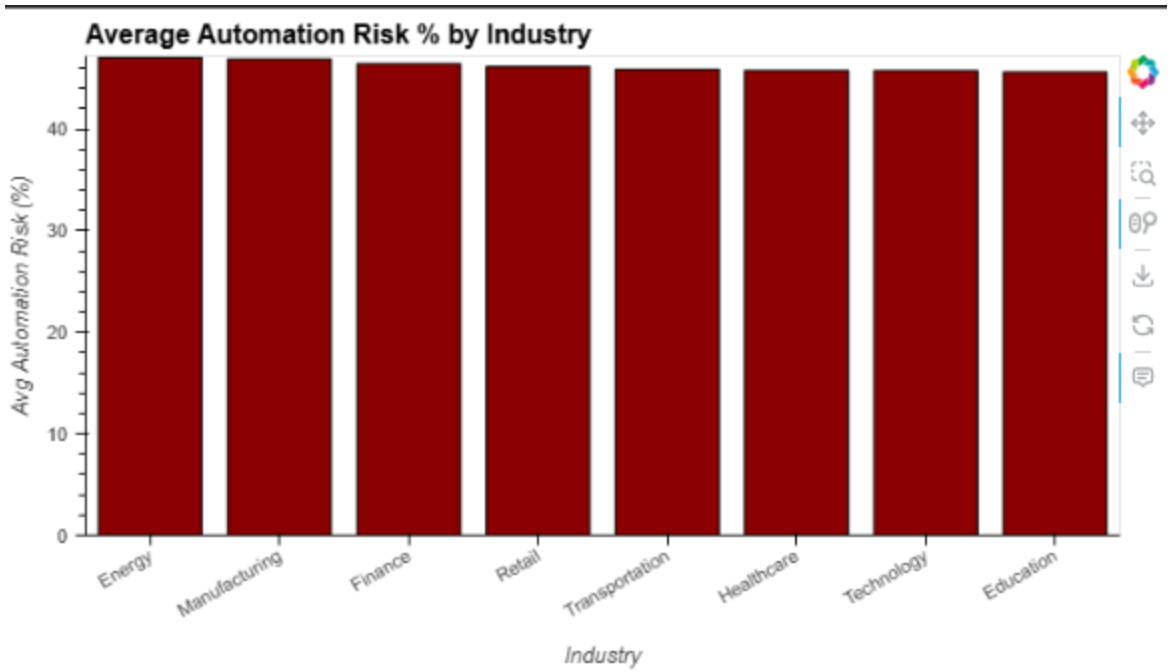
#	Question	pandas Technique	Chart Type
Q1	Avg automation risk % per industry?	groupby, mean, sort_values	Bar Chart
Q2	Countries with highest avg AI replacement score?	column select, groupby, sort	Horizontal Bar
Q3	How does risk category affect salary change?	groupby, agg (mean/median/count)	Bar Chart
Q4	AI adoption & reskilling urgency over years?	groupby year, mean	Line (overlaid)
Q5	Top 10 job roles by salary before AI?	column select, groupby, head(10)	Horizontal Bar
B1	Relationship between risk % and salary change?	scatter on df columns	Scatter Plot
B2	Distribution of AI replacement scores?	histogram	Histogram

III. Source Code (Notebook Cells)

Question 1 — Average Automation Risk % by Industry

Q1 — Bar Chart

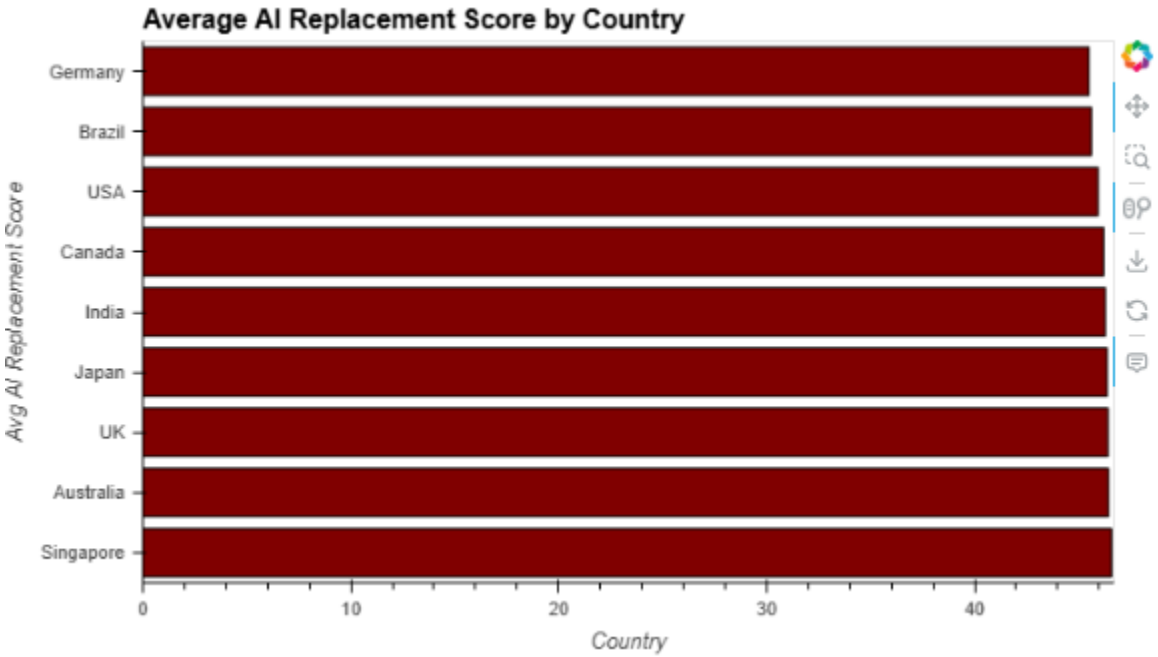
```
q1 = (
    df.groupby("industry")["automation_risk_percent"]
    .mean()
    .round(2)
    .sort_values(ascending=False)
    .reset_index()
)
q1.columns = ["Industry", "Avg Automation Risk (%)"]
q1

q1.hvplot.bar(
    x="Industry",
    y="Avg Automation Risk (%)",
    title="Average Automation Risk % by Industry",
    color="darkred", rot=30, height=400, width=700
)
```



Question 2 — Average AI Replacement Score by Country

Q2 — Horizontal Bar Chart

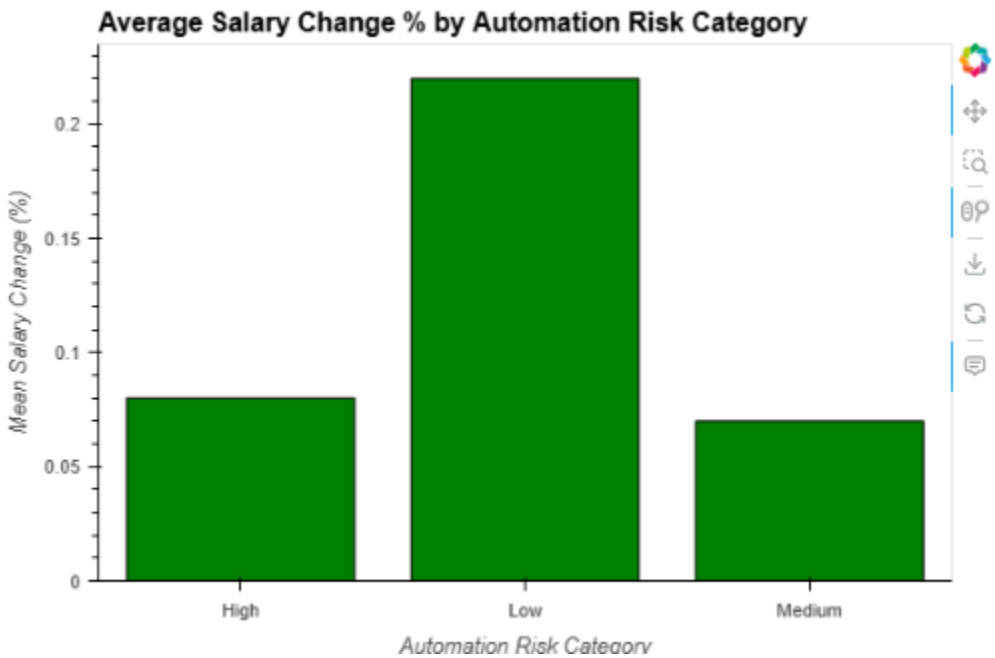
```
q2 = (
    df[["country", "ai_replacement_score"]]
    .groupby("country")["ai_replacement_score"]
    .mean().round(2).sort_values(ascending=False).reset_index()
)
q2.columns = ["Country", "Avg AI Replacement Score"]
q2.hvplot.barh(
    x="Country", y="Avg AI Replacement Score",
    title="Average AI Replacement Score by Country",
    color="maroon", height=400, width=700
)
```



Question 3 — Salary Change % by Automation Risk Category

Q3 — Bar Chart

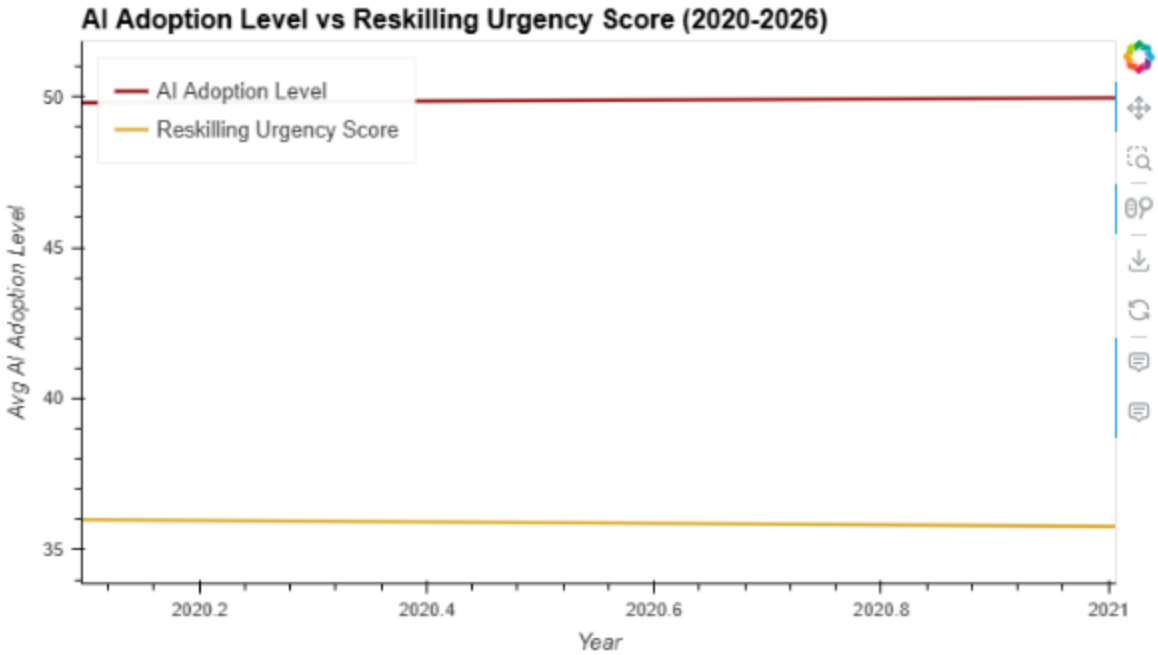
```
q3 = (
    df.groupby("automation_risk_category")["salary_change_percent"]
    .agg(["mean", "median", "count"]).round(2).reset_index()
)
q3.columns = ["Risk Category","Mean Salary Change (%)","Median Salary Change
(%)","Count"]
q3.hvplot.bar(
    x="Risk Category", y="Mean Salary Change (%)",
    title="Average Salary Change % by Automation Risk Category",
    color=["green","orange","red"], height=400, width=600
)
```



Question 4 — AI Adoption & Reskilling Urgency Over Years

Q4 — Line Chart (Overlaid)

```
q4 = (
    df.groupby("year")[["ai_adoption_level","reskilling_urgency_score"]]
    .mean().round(2).reset_index()
)
q4.columns = ["Year","Avg AI Adoption Level","Avg Reskilling Urgency Score"]

chart_adoption = q4.hvplot.line(
    x="Year", y="Avg AI Adoption Level", label="AI Adoption Level",
    color="darkred", line_width=2, markers=True, height=400, width=700
)
chart_reskilling = q4.hvplot.line(
    x="Year", y="Avg Reskilling Urgency Score", label="Reskilling Urgency",
    color="goldenrod", line_width=2, markers=True
)
(chart_adoption * chart_reskilling).opts(
    title="AI Adoption Level vs Reskilling Urgency Score (2020-2026)",
    legend_position="top_left"
)
```



Question 5 — Top 10 Job Roles by Salary Before AI

Q5 — Horizontal Bar Chart

```
q5 = (
    df[["job_role","salary_before_usd"]]
    .groupby("job_role")["salary_before_usd"]
    .mean().round(2).sort_values(ascending=False).head(10).reset_index()
)
q5.columns = ["Job Role","Avg Salary Before AI (USD)"]
q5.hvplot.barh(
    x="Job Role", y="Avg Salary Before AI (USD)",
    title="Top 10 Job Roles by Average Salary Before AI",
    color="maroon", height=450, width=750
)
```

Bonus — Scatter Plot & Histogram

Bonus — Scatter + Histogram

```
# Scatter: Automation Risk % vs Salary Change %
df.hvplot.scatter(
    x="automation_risk_percent", y="salary_change_percent", by="industry",
    title="Automation Risk % vs Salary Change % (by Industry)",
    alpha=0.4, height=450, width=750
)

# Histogram: Distribution of AI Replacement Scores
df.hvplot.hist(
    y="ai_replacement_score", bins=30,
    title="Distribution of AI Replacement Scores",
    color="darkred", alpha=0.8, height=400, width=700
)
```

IV. Areas for Improvement

- **File Path Fix** — In Jupyter Notebooks, __file__ does not exist. The setup cell should use pd.read_csv('ai_job_replacement_2020_2026_v2.csv') directly, assuming the CSV is in the same folder as the notebook.

- **Interactive Filters** — Adding ipywidgets dropdowns for Year and Industry would let users filter the data and regenerate charts dynamically without rerunning cells.

- **More Chart Types** — Adding a heatmap of the correlation matrix or an area chart would increase the variety of visualizations beyond the minimum requirement.

- **Written Interpretation** — Adding a short markdown paragraph after each chart explaining what the chart shows and what conclusions can be drawn would strengthen the academic quality of the notebook.

V. Reflection

Working on the data analysis notebook gave me a much better understanding of how pandas can be used to quickly extract meaningful insights from a large dataset. Using groupby, sort_values, and agg together allowed me to answer complex questions about the data in just a few lines of code, which I found impressive.

hvplot was a new library for me, and I appreciated how simple it made chart creation. Instead of writing many lines of matplotlib configuration code, a single method call like df.hvplot.bar() or df.hvplot.line() produced an interactive chart with pan, zoom, and hover tooltips out of the box. The ability to overlay two line charts using the * operator in Question 4 was particularly useful.

The most important finding was that higher automation risk categories correlate with more negative salary changes, confirming that AI disruption has a real economic impact on workers in high-risk roles. AI adoption levels also showed a clear upward trend from 2020 to 2026, which aligns with reskilling urgency scores also increasing — showing that as AI adoption grows, the pressure on workers to acquire new skills grows with it.

Tolete, Allen Jerrome M. | BS Computer Science | MSEUF Lucena | CP102 M001